

SISTEMA DE BANCA POR INTERNET

“BP”

INTRODUCCION

El presente documento detalla el diseño de la arquitectura de soluciones para el nuevo Sistema de Banca por Internet de la entidad financiera "BP". El objetivo principal de esta propuesta es proveer una plataforma tecnológica robusta, segura y altamente escalable que permita a los clientes acceder a su histórico de movimientos y realizar transferencias con una experiencia de usuario fluida y de muy baja latencia.

Para lograr esto, se ha diseñado una arquitectura Nativa en la Nube (Cloud-Native) basada en Amazon Web Services (AWS), priorizando un enfoque Serverless (sin servidor) y el uso de servicios administrados. Esta decisión estratégica garantiza el cumplimiento de los pilares del Well-Architected Framework, asegurando:

- **Excelencia Operativa y Seguridad:** Mediante el cumplimiento de estrictas normativas bancarias, un flujo de autenticación moderno (OAuth 2.0 con PKCE) e integración biométrica para un onboarding seguro.
- **Rendimiento y Baja Latencia:** Implementando patrones de diseño como Cache-Aside para proteger los sistemas Core heredados y acelerar las consultas.
- **Alta Disponibilidad (HA) y Tolerancia a Fallos:** A través de una arquitectura desacoplada y orientada a eventos (Event-Driven).
- **Optimización de Costos (FinOps):** Un modelo de pago por uso real que elimina el gasto en infraestructura ociosa.

CONSIDERACIONES NORMATIVAS Y DE SEGURIDAD BANCARIA

Como entidad financiera, la arquitectura debe cumplir con estrictas regulaciones (ej. Ley de Protección de Datos Personales, estándares de la Superintendencia de Bancos aplicable, y normativas internacionales):

- **Privacidad y Protección de Datos:** Anonimización y enmascaramiento de PII (Personal Identifiable Information). Cifrado de datos en tránsito (TLS 1.3) y en reposo (AES-256).
- **Seguridad Perimetral y Aplicativa:** Cumplimiento de estándares como PCI-DSS e ISO 27001. Implementación de WAF (Web Application Firewall) en AWS para mitigar ataques del OWASP Top 10.
- **Trazabilidad:** Retención inalterable de logs transaccionales para auditorías regulatorias, garantizando el no repudio de las operaciones.

ESTRATEGIA DE NUBE Y REQUISITOS NO FUNCIONALES

Dada la presión en costos, la necesidad de baja latencia y la exigencia de alta disponibilidad, se selecciona Amazon Web Services (AWS) utilizando un enfoque prioritariamente Serverless.

- **Alta Disponibilidad (HA) y Tolerancia a Fallos:** Despliegue en múltiples Zonas de Disponibilidad (Multi-AZ) dentro de una misma región.
- **Recuperación ante Desastres (DR):** Estrategia Pilot Light hacia una región secundaria, replicando bases de datos críticas asíncronamente para optimizar costos mientras se garantiza un RTO/RPO aceptable.
- **Manejo de Costos (FinOps):** El uso de Serverless (Lambda, API Gateway, DynamoDB) asegura que solo se pague por el cómputo y almacenamiento consumidos.
- **Monitoreo y Autocuración:** Uso de Amazon CloudWatch y AWS X-Ray para trazabilidad distribuida, configurando alarmas que disparen escalado automático (Auto Scaling) ante picos de demanda.

DECISIONES ARQUITECTONICAS

Aplicaciones Frontend (SPA y Movil)

Decisión: Se utilizará React.js para la SPA web y Flutter para la aplicación móvil multiplataforma.

Justificación React (SPA): Se descartan aplicaciones renderizadas en el servidor (SSR) porque una SPA traslada la carga de procesamiento al cliente, reduciendo costos de cómputo en AWS. Se elige React sobre Angular porque su ecosistema facilita la adopción futura de Micro-frontends, permitiendo que distintos equipos del banco desplieguen actualizaciones de forma independiente.

Justificación Flutter (Móvil): Las aplicaciones bancarias requieren alta seguridad que las Web-views no pueden garantizar fácilmente. Se descarta el desarrollo 100% nativo (Kotlin/Swift) por su alto TCO. Flutter compila a código máquina y ofrece rendimiento nativo a 60fps, vital para la fluidez en el onboarding biométrico, manteniendo un único código base.

Arquitectura de Autenticación, Biométrica y Autorización

Decisión: Uso del flujo OAuth 2.0 Authorization Code Flow con PKCE, integrando servicios biométricos en el Identity Provider (IdP).

Justificación del Flujo: El Implicit Flow está deprecado por riesgos de seguridad. Al ser aplicaciones públicas que no pueden guardar un Client Secret, PKCE previene

ataques de interceptación mediante el intercambio de un Code Verifier y un Code Challenge.

Onboarding y Biometría: Para el enrolamiento inicial, se utilizará AWS Rekognition (comparación de selfie vs. documento). Posteriormente, la biometría (huella/rostro) no se envía por la red. Se almacena un Refresh Token en el enclave seguro del dispositivo móvil (Keystore/Secure Enclave). La huella local simplemente desbloquea este enclave para solicitar un nuevo Access Token de forma silenciosa.

Integración y API Gateway

Decisión: Uso de Amazon API Gateway frente a un modelo de microservicios directos.

Justificación: Desacopla a los clientes de los sistemas backend, permitiendo ruteo dinámico, limitación de peticiones (Throttling) para proteger el Core, y validación centralizada de tokens JWT en el borde de la red.

Persistencia, Cache y Core Externo

Decisión: Para reducir la carga sobre el "Core" heredado y mejorar la latencia, se implementará una caché distribuida usando el patrón Cache-Aside con Amazon ElastiCache (Redis).

Justificación del Patrón: Los sistemas Core (Mainframes) son costosos de escalar ante alto volumen de lecturas. Con Cache-Aside, cuando el cliente consulta movimientos, el microservicio busca primero en Redis (Cache Hit). Si no está (Cache Miss), va al Core, devuelve el dato y lo guarda en Redis con un TTL (Time To Live). Esto reduce los MIPS (costos de procesamiento) en el Core heredado y garantiza muy baja latencia.

Diseño del modelo C4

Diagrama de contexto

Este diagrama muestra cómo interactúa el usuario con el sistema de banca y este con los sistemas externos (Core, Notificaciones, IdP).

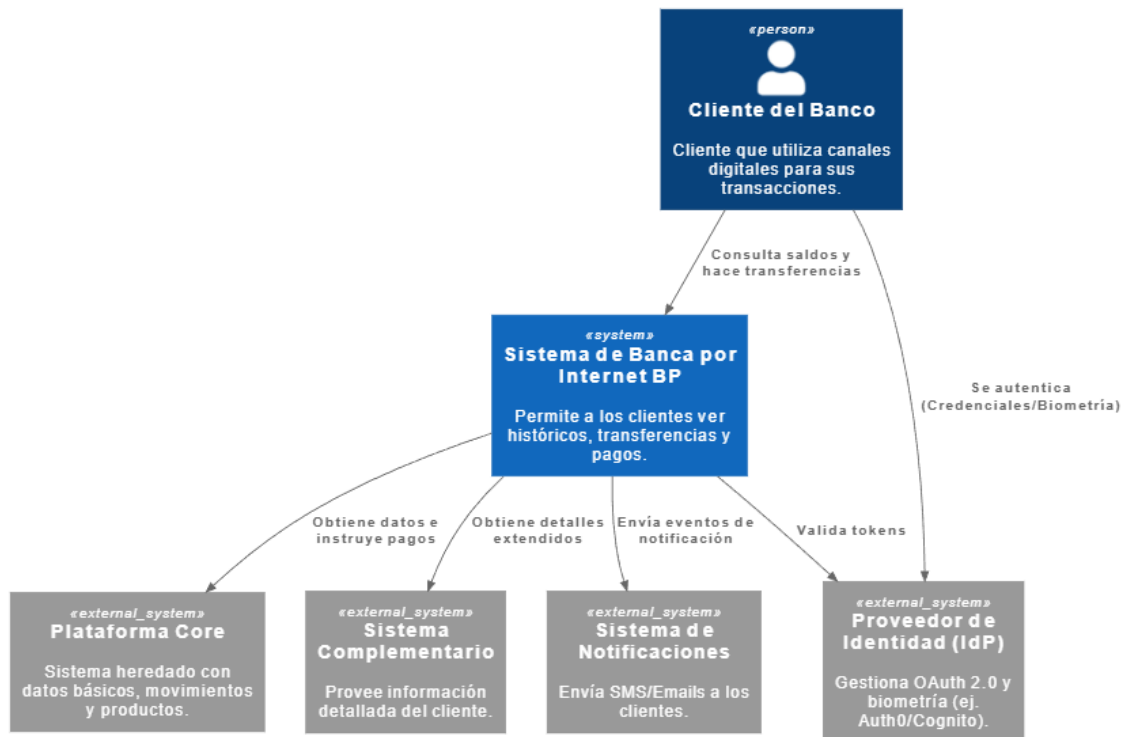


Figura 1: Diagrama de contexto

Diagrama de contenedores

Muestra la arquitectura interna del sistema, diferenciando las aplicaciones frontend, el API Gateway, los microservicios, el bus de eventos y las bases de datos.

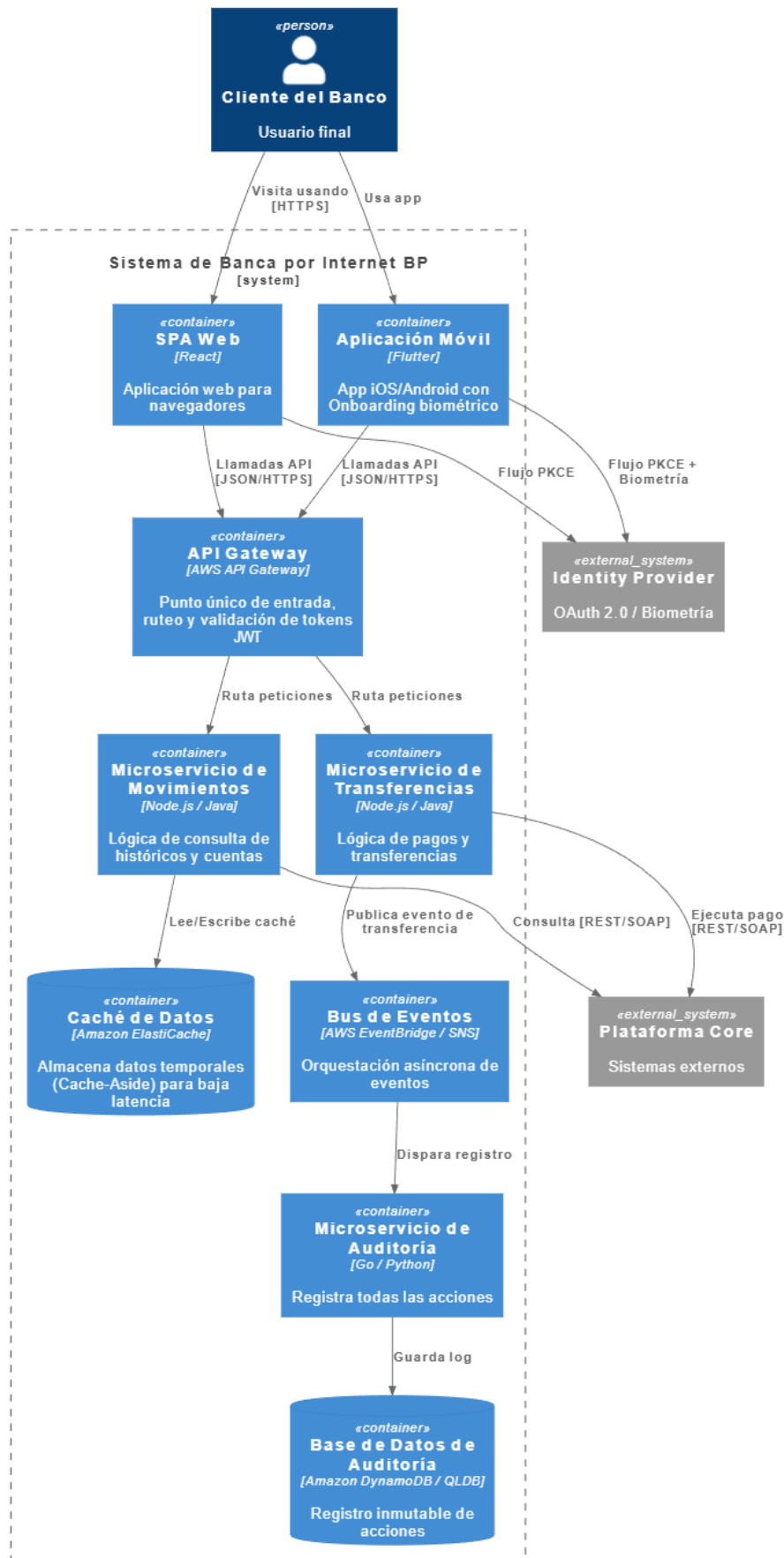


Figura 2: Diagrama de contenedores

Diagrama de componentes

Hace un zoom específico al contenedor del "Microservicio de Transferencias" para ver cómo interactúa internamente aplicando patrones como Circuit Breaker hacia el Core y publicando eventos.

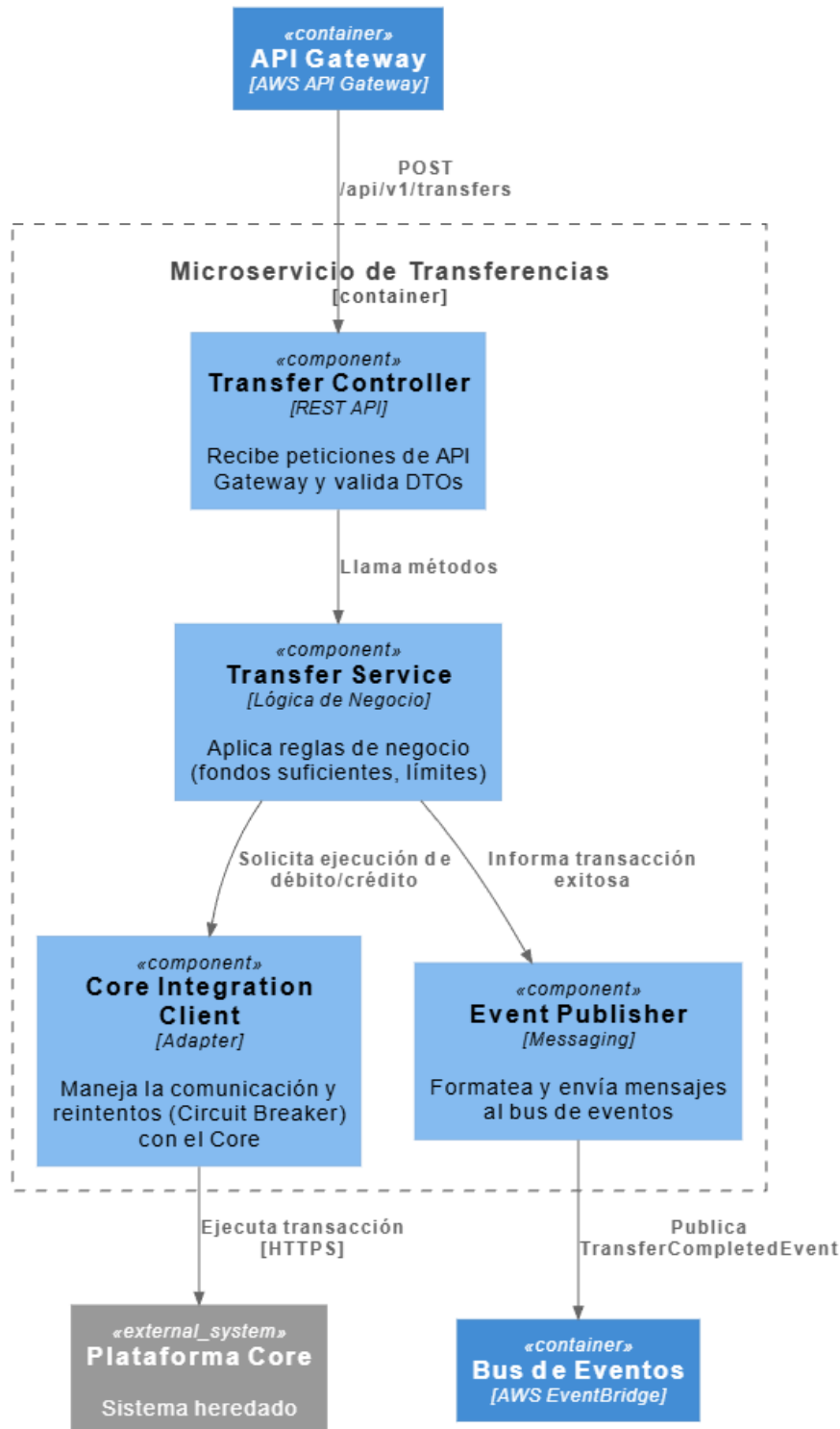


Figura 3: Diagrama de componentes